Übungsblatt 5: Der Single-Cycle-Datenpfad

Mikrocomputertechnik 1

Musterlösung

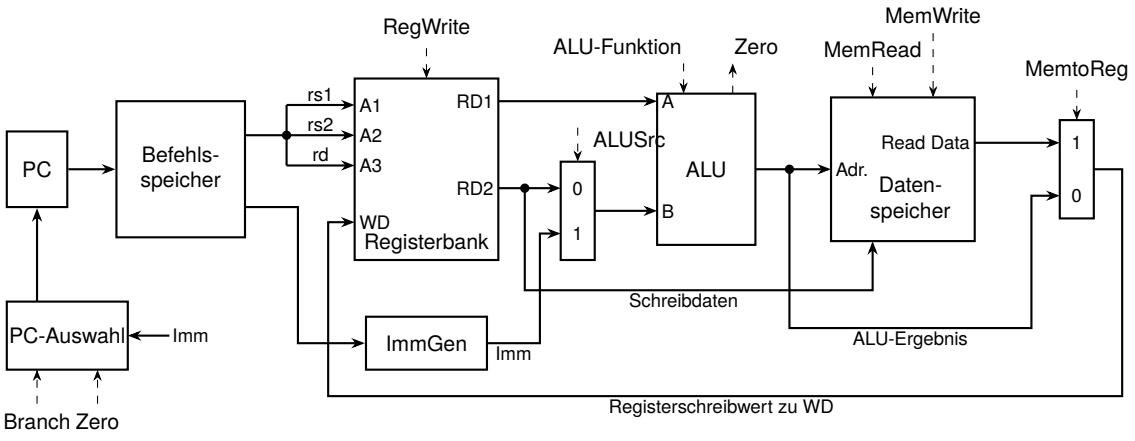
Aufgabe 1: Die Waage rechnet das Nettogewicht (8 Punkte)

Eine Paketwaage an einem Förderband begleitet Sie durch das ganze Blatt. Ihr Prozessor arbeitet nach dem Single-Cycle-Lehrmodell aus Einheit 5. Er zieht vom Bruttogewicht das Gewicht des leeren Kartons (Tara) ab, legt das Nettogewicht ab und vergleicht es mit dem Sollgewicht. Nur ein Paket mit genau dem Sollgewicht erhält ein Etikett mit dem Nettogewicht in a0; sonst wird es ausgeschleust, und a0 wird 0.

Adresse	Inhalt am Anfang	Bedeutung
`0x1000`	1250	Bruttogewicht in Gramm
`0x1004`	50	Tara (leerer Karton)
`0x1008`	0	Nettogewicht, vom Programm geschrieben
`0x100C`	1200	Sollgewicht

Anfang (Fall A): PC = `0x100`, s0 = `0x1000`, a0 = 7, t0 bis t3 = 0.

```
start:          lw   t0, 0(s0)          # 0x100  Bruttogewicht
                lw   t1, 4(s0)          # 0x104  Tara
                sub  t2, t0, t1         # 0x108  Nettogewicht
                sw   t2, 8(s0)          # 0x10C  Netto ablegen
                lw   t3, 12(s0)         # 0x110  Sollgewicht
                beq  t2, t3, gut        # 0x114  Netto = Soll?
ausschleusen:   add  a0, x0, x0         # 0x118  kein Etikett
fertig:         beq  x0, x0, fertig     # 0x11C  Ende
gut:            add  a0, t2, x0         # 0x120  Netto-Etikett
                beq  x0, x0, fertig     # 0x124
```



Durchgezogene Pfeile sind Datenwege, gestrichelte Pfeile Steuersignale; gleichnamige Anschlüsse sind verbunden. Kursvereinbarung: ALUSrc 0 = RD2, 1 = Immediate; MemtoReg 0 = ALU-Ergebnis, 1 = Read Data.

In dieser Aufgabe betrachten Sie nur `sub t2, t0, t1` bei PC = `0x108`. Davor gilt t0 = 1250, t1 = 50 und t2 = 0. Im Maschinenwort `0x406283B3` stehen opcode = `0110011`, funct3 = `000` und funct7 = `0100000` (sub statt add).

1. Ergänzen Sie die Tabelle; die erste Zeile ist ein Beispiel. Tipp: t0 bis t2 sind x5 bis x7. (2 P)

Feld	Bits	Bitmuster	Registernummer	Register
rs1	19:15	`00101`	5	t0
rs2	24:20	`00110`		
rd	11:7	`00111`		

2. Ergänzen Sie die Werte an der Registerbank kurz vor der nächsten steigenden Flanke, dezimal. Die erste Zeile ist ein Beispiel. (3 P)

Port	Bedeutung	Wert vor der Flanke
A1	Leseadresse 1	5
A2	Leseadresse 2	
A3	Schreibadresse	
RD1	gelesener Wert 1	
RD2	gelesener Wert 2	
WD	Schreibdaten	
RegWrite	Schreibfreigabe	

3. Geben Sie t2 und den PC unmittelbar vor und unmittelbar nach der Flanke an. (2 P)
4. Eine kürzere Programmvariante überschreibt das Bruttogewicht: `sub t0, t0, t1`. Welche Werte liegen vor der Flanke an RD1 und WD, und was steht nach der Flanke in t0? (1 P)

Musterlösung

Feld	Bits	Bitmuster	Registernummer	Register
rs2	24:20	`00110`	6	t1
rd	11:7	`00111`	7	t2

1. Je Zeile 1 P.

Port	Bedeutung	Wert vor der Flanke
A2	Leseadresse 2	6
A3	Schreibadresse	7

Port	Bedeutung	Wert vor der Flanke
RD1	gelesener Wert 1	1250 = 0x000004E2
RD2	gelesener Wert 2	50 = 0x00000032
WD	Schreibdaten	1200 = 0x000004B0
RegWrite	Schreibfreigabe	1

2. Je Zeile 0,5 P. A1 bis A3 sind Registernummern, RD1, RD2 und WD dagegen Datenwörter.
3. Vor der Flanke: t2 = 0 und PC = 0x108. Nach der Flanke: t2 = 1200 und PC = 0x10C. Je Wert 0,5 P. t0 und t1 bleiben unverändert.
4. RD1 = 1250, der alte Wert von t0; WD = 1200; nach der Flanke t0 = 1200 (1 P). Die Registerbank übernimmt erst an der Flanke, deshalb entsteht keine endlose Rückkopplung.

Aufgabe 2: Gewichte laden und das Netto ablegen (12 Punkte)

Programm, Datenblock und Anfangszustand wie in Aufgabe 1 (Fall A). Jede Tabellenzeile beschreibt einen Befehl kurz vor seiner Flanke.

1. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel aus Aufgabe 1; „—" bedeutet keine Wirkung. (4 P)

Befehl	Eingang B	ALU-Ergebnis	Datenspeicher	WD-Quelle	Wirkung an der Flanke
`sub t2, t0, t1`	RD2 = 50	1200	—	ALU-Ergebnis	t2 = 1200
`lw t0, 0(s0)`					
`lw t1, 4(s0)`					
`sw t2, 8(s0)`					
`lw t3, 12(s0)`					

2. 1200 ist hexadezimal `0x000004B0`. Ergänzen Sie die Bytes des Netto-Worts nach dem Store; der Kursspeicher ist Little Endian. Beispiel: Das Bruttogewicht 1250 = `0x000004E2` liegt als `E2 04 00 00` an `0x1000` bis `0x1003`. (2 P)

Adresse	`0x1008`	`0x1009`	`0x100A`	`0x100B`
vorher	`00`	`00`	`00`	`00`
nachher				

3. Beim Store wählt ALUSrc das Immediate 8. Auf welchem Weg gelangt der Wert 1200 trotzdem zum Datenspeicher? Warum darf in Ihrer Tabelle beim Store als WD-Quelle ein X stehen? (2 P)
4. Die Waage legt für jedes Paket einen solchen Block aus vier Wörtern ab. Der Block des vorherigen Pakets beginnt 16 Byte tiefer, bei `0x0FF0`. Zwei Zusatzbefehle sollen dessen Netto und Tara laden; s0 bleibt `0x1000`. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. Tipp: Offset = Adresse − s0. (4 P)

Wort	Adresse	Offset	zwölf Offsetbits	ImmGen-Ausgang
Tara, aktuelles Paket	`0x1004`	+4	`0000 0000 0100`	`0x00000004`
Netto, vorheriges Paket				
Tara, vorheriges Paket				

Musterlösung

Befehl	Eingang B	ALU-Ergebnis	Datenspeicher	WD-Quelle	Wirkung an der Flanke
`lw t0, 0(s0)`	Imm = 0	`0x1000`	liest 1250	Read Data	t0 = 1250
`lw t1, 4(s0)`	Imm = 4	`0x1004`	liest 50	Read Data	t1 = 50
`sw t2, 8(s0)`	Imm = 8	`0x1008`	schreibt 1200	X	Wort an `0x1008` = 1200
`lw t3, 12(s0)`	Imm = 12	`0x100C`	liest 1200	Read Data	t3 = 1200

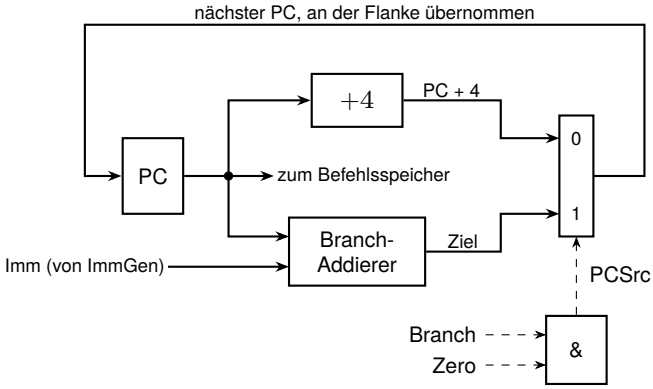
1. Je Zeile 1 P. Bei Loads und Store ist das ALU-Ergebnis die Adresse. Bei den Loads gilt ALUSrc = 1 und MemtoReg = 1, beim Store ALUSrc = 1.
2. Nachher an `0x1008` bis `0x100B`: `B0 04 00 00`. Je Byte 0,5 P. Das niederwertigste Byte liegt an der kleinsten Adresse.
3. rs2 wählt t2 aus; RD2 = 1200 führt über einen Abzweig direkt zu den Schreibdaten des Datenspeichers. ALUSrc wählt nur den zweiten ALU-Operanden (1 P). sw schreibt kein Register, RegWrite ist 0. Der Wert an WD wird nicht übernommen, beide Auswahlen führen zum selben Zustand (1 P).

Wort	Adresse	Offset	zwölf Offsetbits	ImmGen-Ausgang
Netto, vorheriges Paket	`0x0FF8`	–8	`1111 1111 1000`	`0xFFFFFFF8`
Tara, vorheriges Paket	`0x0FF4`	–12	`1111 1111 0100`	`0xFFFFFFF4`

4. Je Zeile 2 P: Adresse und Offset 1 P, Offsetbits und ImmGen-Ausgang 1 P. Die Befehle lauten etwa `lw t4, -8(s0)` und `lw t5, -12(s0)`. Kontrolle: `0x00001000` + `0xFFFFFFF8` ergibt modulo 2^{32} die Adresse `0x00000FF8`.

Aufgabe 3: Etikett oder ausschleusen? (10 Punkte)

Die PC-Auswahl des Kursmodells: Ein eigener Branch-Addierer bildet aus dem alten PC und dem Byteoffset das Sprungziel. ImmGen liefert bei beq den fertigen, vorzeichenerweiterten Byteoffset einschließlich Bit 0. Die ALU subtrahiert RD2 von RD1 und meldet Zero = 1, wenn das Ergebnis null ist. Es gilt PCSrc = Branch ∧ Zero.



Fall A ist der Anfangszustand aus Aufgabe 1. In Fall B ist das Paket zu leicht: Bruttogewicht 1230, alles andere wie in Fall A. Das Nettogewicht wird dann 1180.

1. Ergänzen Sie die Tabelle für die drei beq-Befehle des Programms. Die erste Zeile ist ein Beispiel. Tipp: Der Byteoffset ist Ziel minus Adresse des beq selbst, nicht minus PC + 4. (3 P)

PC	Befehl	Byteoffset	ImmGen-Ausgang	Ziel	PC + 4
`0x11C`	`beq x0, x0, fertig`	0	`0x00000000`	`0x11C`	`0x120`
`0x114`	`beq t2, t3, gut`				
`0x124`	`beq x0, x0, fertig`				

2. Ergänzen Sie die Tabelle; die Befehle stehen im Programm aus Aufgabe 1. Die erste Zeile ist ein Beispiel. (3 P)

Befehl	Fall	RD1	RD2	ALU-Ergebnis	Zero	Branch	PCSrc	nächster PC
sub bei `0x108`	A	1250	50	1200	0	0	0	`0x10C`
beq bei `0x114`	A							
beq bei `0x114`	B							
beq bei `0x124`	A							

3. Auf der Waage liegt ein leerer Karton: Bruttogewicht = Tara = 50. `sub t2, t0, t1` liefert 0, also Zero = 1. Springt der Prozessor? Welcher PC folgt? (1 P)
4. Geben Sie für Fall A und Fall B die Adressen der ausgeführten Befehle ab `0x114` bis zum ersten Erreichen von `fertig` an, dazu a0 und das Netto-Wort an `0x1008` bei `fertig`. (2 P)
5. Ein fehlerhafter Entwurf schiebt den ImmGen-Ausgang vor dem Branch-Addierer noch einmal um eine Stelle nach links. Wohin springt `beq t2, t3, gut` in Fall A? (1 P)

Musterlösung

PC	Befehl	Byteoffset	ImmGen-Ausgang	Ziel	PC + 4
0x114	`beq t2, t3, gut`	+12	0x0000000C	0x120	0x118
0x124	`beq x0, x0, fertig`	−8	0xFFFFFFF8	0x11C	0x128

1. Je Zeile 1,5 P. Beide Kandidaten werden immer berechnet; erst PCSrc wählt.

Befehl	Fall	RD1	RD2	ALU-Ergebnis	Zero	Branch	PCSrc	nächster PC
beq bei 0x114	A	1200	1200	0	1	1	1	0x120
beq bei 0x114	B	1180	1200	−20	0	1	0	0x118
beq bei 0x124	A	0	0	0	1	1	1	0x11C

2. Je Zeile 1 P. In Fall B ist die Differenz −20 = 0xFFFFFFEC. Branch ist bei jedem beq 1, auch wenn nicht gesprungen wird.
3. Nein. Bei sub ist Branch = 0, also PCSrc $= 0 \wedge 1 = 0$; der nächste PC ist 0x10C (1 P). t2 wird 0.
4. Fall A: 0x114, 0x120, 0x124, 0x11C; a0 = 1200, Netto-Wort 1200 (1 P). Fall B: 0x114, 0x118, 0x11C; a0 = 0, Netto-Wort 1180 (1 P).
5. Der Offset wird zu 24: $0\text{x}114 + 24 = 0\text{x}12\text{C}$, hinter dem letzten Befehl bei 0x124 (1 P). Der fertige Byteoffset darf nicht erneut verdoppelt werden.

Aufgabe 4: Fehlersuche: Das Netto-Wort bleibt null (10 Punkte)

Ein Prototyp der Waage arbeitet Fall A mit dem Anfangszustand aus Aufgabe 1 ab. Bei `fertig` stehen alle Register richtig, auch a0 = 1200: Das Paket erhält sein Etikett. Die Anzeige liest jedoch das Netto-Wort an `0x1008` und zeigt 0. Ein Speicherauszug zeigt das Wort 1200 an der Adresse `0x14B0`.

1. Welcher Befehl hätte das Netto-Wort schreiben sollen? Zerlegen Sie `0x14B0` in s0 und einen zweiten Summanden. Woher stammt dieser? Tipp: 1200 = `0x4B0`. (2 P)
2. Welche Auswahlstelle war beim Store falsch eingestellt? Geben Sie das Steuersignal mit seinem falschen und seinem richtigen Wert an. (2 P)
3. Warum erhält das Paket trotzdem richtig ein Etikett, und warum bleiben alle Register richtig? (2 P)
4. Wie verhält sich der Prototyp in Fall B (Bruttogewicht 1230)? Geben Sie die beschriebene Adresse, die Anzeige des Netto-Worts und a0 an. Tipp: 1180 = `0x49C`. (2 P)
5. Geben Sie die richtige Zeile für sw an: ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch und ALU-Funktion. Ein Kollege möchte stattdessen MemtoReg umstellen. Warum hilft das nicht? (2 P)

Musterlösung

1. `sw t2, 8(s0)` bei `0x10C` (1 P). `0x14B0` = `0x1000` + `0x4B0`, also s0 plus 1200. Der zweite Summand ist t2, der Wert von RD2, statt des Offsets 8 (1 P).
2. Der MUX vor ALU-Eingang B. ALUSrc war 0 (RD2), richtig ist 1 (Immediate). Je 1 P. Die Schreibdaten kamen trotzdem richtig aus RD2, nur die Adresse war falsch.
3. beq vergleicht die Register t2 und t3, nicht das Netto-Wort im Speicher (1 P). sw hat RegWrite = 0 und verändert nur ein Speicherwort; Loads und sub waren richtig gesteuert (1 P).
4. `0x1000` + `0x49C` = `0x149C` erhält 1180 (1 P). Das Netto-Wort an `0x1008` zeigt weiter 0; a0 = 0, das Paket wird richtig ausgeschleust (1 P).
5. sw: ALUSrc = 1, MemtoReg = X, RegWrite = 0, MemRead = 0, MemWrite = 1, Branch = 0, ALU-Funktion ADD (1 P, anteilig). MemtoReg wählt nur den Registerschreibwert WD. Beim Store ist RegWrite = 0; Adresse und Schreibdaten hängen nicht von MemtoReg ab (1 P).

Aufgabe 5: Wie schnell darf die Waage takten? (10 Punkte)

Die folgenden Verzögerungen sind erfundene Lehrwerte, keine Messwerte.

Baustein	Zeit in ns
Clock-to-Q (PC und Register)	0,2
Befehlsspeicher lesen	1,5
Registerlesen	0,8
ImmGen	0,6
ALU (Ergebnis und Zero)	1,2
Datenspeicher lesen	2,0
Rückschreib-MUX	0,3
Branch-Addierer	0,9
UND-Gatter und PC-MUX zusammen	0,4
Setup	0,2

Registerlesen und ImmGen laufen parallel nach dem Befehlsspeicher. Steuersignale und der MUX vor ALU-Eingang B sind rechtzeitig gültig und kosten keine eigene Zeit. Der Store-Weg ist nicht länger als der Load-Weg. Clock-Skew und Reserve werden vernachlässigt.

1. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. Die Wege zum Register enden mit dem Setup, die beiden beq-Zeilen an der PC-Auswahl. (4 P)

Weg	Bausteine nacheinander	Zeit
sub bis zum Register	Clock-to-Q 0,2 + Befehlsspeicher 1,5 + Registerlesen 0,8 + ALU 1,2 + Rückschreib-MUX 0,3 + Setup 0,2	4,2 ns
lw bis zum Register		
beq: Zero gültig		
beq: Sprungziel gültig		

2. An der PC-Auswahl treffen sich beide beq-Wege. Danach folgen UND-Gatter und PC-MUX sowie das Setup des PC. Welche Mindestzeit verlangt beq? Warum addieren Sie die beiden Wege nicht? (2 P)
3. Bestimmen Sie die gemeinsame Taktperiode und die ideale Maximalfrequenz. Wie viel Zeit jeder Periode bleibt bei sub ungenutzt? (2 P)
4. Ausbaustufe: Ein schnelleres Förderband verlangt 200 MHz. Wie lange darf das Lesen des Datenspeichers dann höchstens dauern? Passen sub und beq noch in die Periode? (2 P)

Musterlösung

Weg	Bausteine nacheinander	Zeit
lw bis zum Register	Clock-to-Q 0,2 + Befehlsspeicher 1,5 + Registerlesen 0,8 + ALU 1,2 + Datenspeicher 2,0 + Rückschreib-MUX 0,3 + Setup 0,2	6,2 ns
beq: Zero gültig	Clock-to-Q 0,2 + Befehlsspeicher 1,5 + Registerlesen 0,8 + ALU 1,2	3,7 ns
beq: Sprungziel gültig	Clock-to-Q 0,2 + Befehlsspeicher 1,5 + ImmGen 0,6 + Branch-Addierer 0,9	3,2 ns

1. lw 2 P (Folge 1 P, Zeit 1 P), die beiden beq-Zeilen je 1 P. Beim Load liegt der Datenspeicher zwischen ALU und Rückschreib-MUX.
2. $T_{\text{beq}} = \max(3{,}7;\ 3{,}2) + 0{,}4 + 0{,}2 = 4{,}3$ ns (1 P). Vergleich und Zielrechnung laufen parallel; die PC-Auswahl wartet nur auf den späteren Wert (1 P).
3. Der längste Weg ist lw: $T_{\text{clk}} = 6{,}2$ ns, $f_{\text{max}} = 1/6{,}2$ ns ≈ 161 MHz (1 P). sub ist nach 4,2 ns fertig, wird aber erst an derselben nächsten Flanke übernommen: 2,0 ns bleiben ungenutzt (1 P).
4. 200 MHz bedeuten 5,0 ns. Ohne Datenspeicher braucht der Load-Weg 4,2 ns, also darf das Lesen höchstens $5{,}0 - 4{,}2 = 0{,}8$ ns dauern (1 P). sub mit 4,2 ns und beq mit 4,3 ns passen in 5,0 ns (1 P).

Aufgabe 6: Die Paketwaage in Ripes prüfen

Ab `start` steht das Programm aus Aufgabe 1. Davor lädt `la` die Adresse des Datenblocks `paket` nach s0, und `li` setzt a0 auf 7.

```
.data
.align 2
paket: .word 1250, 50, 0, 1200
.text
.globl main
main:
    la   s0, paket
    li   a0, 7
start:
    lw   t0, 0(s0)
    lw   t1, 4(s0)
    sub  t2, t0, t1
    sw   t2, 8(s0)
    lw   t3, 12(s0)
    beq  t2, t3, gut
ausschleusen:
    add  a0, x0, x0
fertig:
    beq  x0, x0, fertig
gut:
    add  a0, t2, x0
    beq  x0, x0, fertig
```

1. Öffnen Sie Ripes, installiert oder im Browser unter ripes.dk. Geben Sie das Programm im Tab *Editor* ein; Ripes übersetzt es automatisch. Wählen Sie über *Select Processor* den *RISC-V Single Cycle Processor* für RV32I ohne Erweiterungen unter *ISA Exts.* und das Layout *Extended*. Protokollieren Sie Version und Modell.
2. Klicken Sie *Reset* und dann einzeln *Clock*. Die Befehlsansicht im Tab *Processor* zeigt in der Spalte *Addr* die Adresse und in der Spalte *Stage* den gerade ausgeführten Befehl. Notieren Sie die Adressen der drei beq-Befehle und prüfen Sie die Byteoffsets aus Aufgabe 3. Lesen Sie bei `start` in der Ansicht *Registers* die Adresse B des Datenblocks in s0 ab; B ersetzt `0x1000`.
3. Halten Sie vor jedem Befehl ab `start` an. Lesen Sie vor der Clock ab: bei `sub` die beiden ALU-Eingänge und das ALU-Ergebnis, bei den Loads Adresse und Read Data des Datenspeichers, beim Store Adresse und Schreibdaten. Vergleichen Sie mit Aufgabe 1 und 2. Prüfen Sie nach der Clock Register und PC und nach dem Store im Tab *Memory* das Wort an B + 8; *Go to register* mit s0 springt zum Datenblock.
4. Lesen Sie bei `beq t2, t3, gut` die Ausgänge der Komponenten `alu` und `branch` ab. Welche Komponente vergleicht, welche bildet das Sprungziel? Vergleichen Sie mit dem Kursmodell aus Aufgabe 3.
5. Ändern Sie im Datenabschnitt das Bruttogewicht auf 1230 (Fall B). Sagen Sie Netto-Wort, Pfad und a0 voraus, bevor Sie das Programm neu starten, und prüfen Sie dann.

Die Initialisierung vor `start` und die Schleife bei `fertig` gehören nicht zum untersuchten Ablauf.

Signale in Ripes ablesen. Fahren Sie mit der Maus über einen Anschluss: Ein Tooltip zeigt Name und Wert. *Display signal values* blendet alle Werte ein, Rechtsklick auf einen Anschluss und *Show value* einen einzelnen. Ein Multiplexer markiert den gewählten Eingang mit einem grünen Punkt; 1-Bit-Leitungen sind bei 1 grün, bei 0 grau. Ripes zeigt Werte oft hexadezimal: 1250 = `0x4E2`, 50 = `0x32`, 1200 = `0x4B0`, 1180 = `0x49C`.

Kursmodell	Komponente in Ripes	Anschlüsse
RD1, RD2	`registerFile`	`r1_out`, `r2_out`
WD, RegWrite	`registerFile`	`data_in`, `wr_en`
ImmGen-Ausgang	`immediate`	`imm`
ALU-Eingänge A und B, Ergebnis	`alu`	`op1`, `op2`, `res`
Adresse, Schreibdaten, Read Data	`data_mem`	`addr`, `data_in`, `data_out`
MUX vor ALU-Eingang B (ALUSrc)	`alu_op2_src`	0 = Register, 1 = Immediate
Rückschreib-MUX (MemtoReg)	`reg_wr_src`	0 = Speicher, 1 = ALU, 2 = PC + 4
Zero	`branch`	`res`

Drei Abweichungen vom Kursmodell sollten Sie kennen:

- Bei `beq` berechnet die ALU das Sprungziel PC + Immediate: Der MUX `alu_op1_src` wählt den PC statt RD1, `alu_op2_src` das Immediate. Einen eigenen Branch-Addierer gibt es nicht.
- Den Vergleich übernimmt die Komponente `branch`; ihr Ausgang `res` entspricht Zero.
- Der Rückschreib-MUX ist umgekehrt codiert und hat einen dritten Eingang für PC + 4.

Die Namen gelten für aktuelle Ripes-Versionen. Notieren Sie Abweichungen Ihrer Version im Protokoll.

Musterlösung

Die absoluten Befehlsadressen hängen davon ab, wie viele Befehle `la` und `li` erzeugen. Die Byteoffsets bleiben +12, 0 und −8, weil sie relativ zur Adresse des jeweiligen beq gelten.

Erwartete Ripes-Werte in Fall A laut Ripes-Quellcode; `op1`, `op2` und `res` gehören zur Komponente `alu`:

Befehl	`op1`	`op2`	`res`	Datenspeicher	`reg_wr_src`
`lw t0, 0(s0)`	B	0	B	liest 1250	0 (Speicher)
`lw t1, 4(s0)`	B	4	B + 4	liest 50	0 (Speicher)
`sub t2, t0, t1`	1250	50	1200	—	1 (ALU)
`sw t2, 8(s0)`	B	8	B + 8	schreibt 1200	1, ohne Wirkung
`lw t3, 12(s0)`	B	12	B + 12	liest 1200	0 (Speicher)
`beq t2, t3, gut`	PC des beq	12	Adresse von `gut`	—	1, ohne Wirkung

Bei `beq t2, t3, gut` liefert `branch` den Wert 1, weil 1200 = 1200; die ALU addiert PC und Offset. Im Kursmodell vergleicht dagegen die ALU, und der Branch-Addierer bildet das Ziel. Die Befehlswirkung ist dieselbe.

Erwartung bei `fertig` in Fall A: t0 = 1250, t1 = 50, t2 = 1200, t3 = 1200, a0 = 1200; Wort an B + 8 = 1200 = `0x4B0`. Pfad über `gut`; der beq danach springt 8 Byte zurück nach `fertig`.

Fall B: t0 = 1230, t2 = 1180, Wort an B + 8 = 1180 = `0x49C`. `branch` liefert 0, der Branch ist nicht genommen; der Pfad führt über `ausschleusen`, a0 = 0. Das sind Modellvorhersagen; Portnamen und Codierungen der eigenen Ripes-Version sind im Protokoll festzuhalten.